VIZZIO Deployment Platform

Full Documentation Guide

Technical · Deployment · Administration · Launcher · Operations · Handover

Deployment prerequisites

Deployments may include an optional `prerequisites.json` file beside `Launch.bat`. The launcher evaluates this file before running the deployment.

Example:

```
{
  "prerequisites": [
    {
      "name": "Node.js",
      "check": {
        "type": "command",
        "command": "node",
        "args": ["--version"]
      }
    },
    {
      "name": "npm",
      "check": {
        "type": "command",
        "command": "npm",
        "args": ["--version"]
      }
    }
  ],
  "requiredPorts": [
    { "name": "Frontend", "port": 3000 },
    { "name": "Local API", "port": 8080 }
  ]
}
```

Supported check types:

- `command`: runs a command and passes if it exits with code 0.
- `file`: passes if the declared file or folder exists.
- `env`: passes if an environment variable exists, or matches `equals`.

If a missing prerequisite has an `installer`, the launcher asks the user before running it. Installer paths are relative to the installed deployment folder.

Port conflict checks:

- `ports`: a simple array of port numbers, such as `[3000, 8080]`.
- `requiredPorts`: named ports used for readable manifests.

The launcher checks only the TCP ports declared by the deployment being launched. If any are already listening, launch is blocked with a clear error so the user can close the conflicting app first. Multiple deployments can run at the same time as long as their declared ports do not conflict.

Hosted PC prerequisites

Use this checklist when the backend runs on a supervisor Windows PC and is exposed through Cloudflare Tunnel.

Required software

- Node.js LTS
- PostgreSQL
- 7-Zip CLI on the backend PC for `.7z` package validation
- .NET 8 SDK on the build/admin PC for launcher builds
- Inno Setup 6 if building the Windows launcher installer
- Cloudflare Tunnel configured to forward the public hostname to the backend port

Backend environment

Set production values in `backend/.env` before exposing the backend:

```
DATABASE_URL=postgresql://postgres:strong-password@localhost:5432/vizzio
NODE_ENV=production
JWT_SECRET=replace-this-with-a-long-random-secret
DOWNLOAD_SECRET=replace-this-with-a-different-long-random-secret
DOWNLOAD_MANAGER_SECRET=replace-this-with-a-third-long-random-secret
PORT=4000
PACKAGE_ROOT=C:\VIZZIO\packages
PACKAGE_UPLOAD_ROOT=C:\VIZZIO\uploads
DOWNLOAD_ROOT=C:\VIZZIO\packages
DOWNLOAD_DELIVERY_MODE=node
DOWNLOAD_ACCEL_PREFIX=/_vizzio_downloads
PACKAGE_UPLOAD_MAX_BYTES=85899345920
UPLOAD_SESSION_RETENTION_MS=604800000
HTTP_REQUEST_TIMEOUT_MS=0
HTTP_HEADERS_TIMEOUT_MS=60000
HTTP_KEEP_ALIVE_TIMEOUT_MS=5000
LAUNCHER_ERROR_REPORT_ROOT=C:\VIZZIO\launcher-error-reports
LAUNCHER_LATEST_VERSION=0.1.0
LAUNCHER_DOWNLOAD_URL=https://example.com/VIZZIO-Launcher-Setup-0.1.0.exe
LAUNCHER_RELEASE_NOTES=Initial hosted release.
LAUNCHER_UPDATE_REQUIRED=false
ENABLE_DEMO_USERS=false
```

Do not use the default JWT or download secrets when the backend is reachable through Cloudflare Tunnel.

Before starting the hosted backend after pulling updates, apply database migrations and regenerate Prisma Client:

```
cd C:\Users\User\Desktop\VIZZIO_Deployment_Platform\backend
npx prisma migrate deploy
npx prisma generate
```

`PACKAGE_ROOT` limits server-side archive and staging-folder paths. Put files there only when registering by server path. Browser uploads can be selected from any admin computer location and are copied into backend storage after validation.

Use `DOWNLOAD_DELIVERY_MODE=node` when the Windows backend streams files directly. If Nginx is added, change it to `nginx` and configure `PACKAGE_ROOT` and `DOWNLOAD_ROOT` to the same Nginx-backed private directory (or otherwise ensure every downloadable package is inside both roots).

7z backend setup

Install 7-Zip on the backend PC and make sure this folder is on the system `PATH`:

```
C:\Program Files\7-Zip
```

Restart the backend terminal or Windows service, then verify:

```
7z i
```

The backend accepts ZIP and 7z package sources, but each package must contain a launch `.bat` at the archive root or inside the only top-level folder. ZIP validation is built in. 7z validation requires `7z` or `7za` on the backend PC.

Large Unreal deployments are expected to be tens of GiB. For server staging folders, the backend creates a generated `.7z` package when 7-Zip is available. Install 7-Zip on the backend PC before publishing 50-60 GiB deployments.

Launcher 7z extraction

User machines do not need to install 7-Zip manually. The launcher installer must bundle `7z.exe` or `7za.exe` beside `Launcher.exe`; otherwise `.7z` downloads will finish but installation will fail during extraction.

```
.\scripts\build_launcher_installer.ps1 -Version 0.1.0 -SevenZipPath "C:\Program Files\7-Zip\7z.exe"
```

The build script auto-detects `launcher\tools\7za.exe`, `launcher\tools\7z.exe`, the standard Windows 7-Zip install path, or `PATH`. If no extractor is available, installer creation fails so a broken launcher is not shipped.

At runtime the launcher checks beside `Launcher.exe` first, then falls back to `PATH`.

Admin readiness check

In the web admin panel, open `Settings > Server` and run `Test Connection`. The readiness check reports:

- Database URL configuration
- JWT and download secret configuration
- Package root accessibility

- Upload storage accessibility
- Download root configuration
- 7z / 7za availability
- Backend port

Fix any required error before giving launcher users access through the tunnel.

Frontend and launcher URLs

When building or hosting the frontend, set:

```
VITE_API_BASE=https://your-cloudflare-hostname.example/api
VITE_DOWNLOAD_BASE=https://your-cloudflare-hostname.example/downloads
```

For the Windows launcher, point it at the Cloudflare backend URL in the Server URL field for a diagnostic session, or stamp it into the production build:

```
.\scripts\build_launcher_installer.ps1 `
  -Version "0.1.0" `
  -ApiBaseUrl "https://your-cloudflare-hostname.example/api" `
  -InnoCompiler "C:\Program Files (x86)\Inno Setup 6\ISCC.exe"
```

See `docs/configuration-and-production-handover.md` for the complete local acceptance, production promotion, and supervisor sign-off checklist.

Architecture Overview

Core Components

- **Admin Web Panel**
  - React SPA built with Vite
  - User and deployment management UI
  - Communicates with the backend API
- **Backend API**
  - Node.js + Express
  - PostgreSQL for persistent storage
  - JWT authentication and bcrypt password hashing
  - Deployment and version management endpoints
- **Launcher Client**
  - C# .NET 8 WPF application
  - Authenticates with backend API
  - Downloads build packages with resumable support, adaptive 4-16 stream selection, and jittered retry backoff
  - Installs versions side-by-side
- **File Delivery System**
  - Nginx serving large Unreal build files
  - HTTP range request support for resumable downloads
  - Backend-authorized `X-Accel-Redirect` delivery for private server files
  - Static file delivery and reverse proxy for backend API

Deployment Flow

1. Admin copies large package files onto the server and registers the server file path on a deployment version.

2. Backend validates the package, stores metadata, and later issues scoped download-manager tokens.
3. Launcher authenticates the user and requests available deployments.
4. Launcher creates a download session and requests the tokenized backend file URL.
5. Backend validates the token and access. It either streams byte ranges directly or returns an internal `X-Accel-Redirect` for Nginx.
6. Launcher downloads resumable parallel ranges, verifies SHA-256, and extracts the package.
7. User can launch or uninstall the installed version.

VIZZIO Diagrams

This document centralizes project diagrams for architecture and flow reviews.

1. System Architecture

```
graph TD
    A["Admin Web Panel<br/>React + Vite"] -->|REST /api| B["Backend API<br/>Node.js + Express"]
    L["Windows Launcher<br/>.NET 8 WPF"] -->|REST /api| B
    B -->|Prisma| D[(PostgreSQL)]
    L -->|Tokenized range request| B
    B -->|Node stream or X-Accel-Redirect| F[Nginx or Node Delivery]
    F --> S[(Package Storage)]
    B --> S
```

2. End-to-End User Flow

```
flowchart LR
    A1[Admin logs in] --> A2[Create deployment]
    A2 --> A3["Add version<br/>upload archive or register server archive or prepare staging
folder"]
    A3 --> A4[Set channel and release state]
    A4 --> A5[Grant group access]

    U1[Launcher user logs in] --> U2[Fetch accessible items]
    U2 --> U3{Start download?}
    U3 -->|Yes| U4[Create session and token]
    U4 --> U5[Parallel range download with resume]
    U5 --> U6[SHA-256 verification]
    U6 --> U7[Extraction]
    U7 --> U8[Installed state]
    U3 -->|No| U9[Refresh later]
```

3. Admin Use Cases

```
flowchart TB
    Admin((Admin))
    UC1[Login]
    UC2[Manage Users]
    UC3[Manage Groups]
    UC4[Manage Deployments]
    UC5[Manage Versions]
    UC6[Grant/Revoke Access]
    UC7[Review Download Logs]
    UC8[Maintenance and Settings]

    Admin --> UC1
    Admin --> UC2
    Admin --> UC3
    Admin --> UC4
    Admin --> UC5
    Admin --> UC6
    Admin --> UC7
    Admin --> UC8
```

4. Launcher User Use Cases

```
flowchart TB
    User((Launcher User))
    U1[Login]
    U2[View Deployments]
    U3[Start Download]
    U4[Pause Resume Cancel]
    U5[Verify Package Integrity]
    U6[Install Version]
    U7[Open Installed Folder]
    U8[Change Install Root]
    U9[Sign Out]

    User --> U1
    User --> U2
    User --> U3
    User --> U4
    User --> U5
    User --> U6
    User --> U7
    User --> U8
    User --> U9
```

5. Download Pipeline Sequence

```
sequenceDiagram
    participant U as Launcher User
    participant L as Launcher App
    participant B as Backend API
    participant F as File Delivery

    U->>L: Start download
    L->>B: Create download session
    B-->>L: Metadata and token
    L->>F: Range requests with token
    B->>B: Validate token, session, access, and path
    B-->>F: Stream directly or authorize internal redirect
    F-->>L: Byte ranges
    L->>L: Merge and checksum
    L->>L: Extract and mark installed
    L->>B: Write activity log
```

Admin User Guide

1. Purpose

This guide explains how platform administrators use the VIZZIO Deployment Platform Admin Web Panel to manage users, groups, deployment access, deployment families, versions, notifications, logs, launcher reports, and platform settings.

2. Access and Login

1. Open the Admin Web Panel URL.
2. Enter an administrator username or email address.
3. Enter the password.
4. Select **Sign In**.

Expected behavior:

- Invalid credentials return an authentication error.
- Disabled managed accounts cannot sign in.
- Repeated failed attempts from the same client can cause temporary rate limiting.
- An invalid or expired session returns the administrator to the login page.

3. Admin Navigation

The sidebar contains:

- **Overview**
- **Deployments**
- **Versions**
- **Users & Permissions**
- **Download Logs**
- **Launcher Reports**
- **Settings**

- **Help & Docs**

Groups are managed inside **Users & Permissions**. Notifications are available from the notification bell in the top navigation. A full Notifications page also exists at `/notifications`, although the current sidebar and bell menu do not link to it.

4. Dashboard Overview

Use **Overview** as the daily starting point. It shows:

- Deployment, version, released stable-version, and group counts
- The number of active users
- Quick actions for creating deployments, registering versions, managing access, and reviewing launcher reports
- **Needs Attention** items for deployments without versions, deployments without a released version, and groups without deployment access
- Recent deployment, version, and group activity

Use **Needs Attention** as a setup checklist before asking launcher users to test.

5. User Management

Open **Users & Permissions** to search and filter users, manage accounts, and assign group-based deployment access. The user list is paginated at 6 users per page. The shared search matches user names, usernames, email addresses, group names, and group membership.

5.1 Create User

1. Go to **Users & Permissions**.
2. Select **New User**.
3. Enter the user's name, username, and email address.
4. Select the role: **Admin** or **User**.
5. Select the initial status: **Active** or **Inactive**.
6. Optionally select one or more groups.
7. Enter a temporary password, or leave **Temporary Password** blank to generate one.
8. Select **Save User**.

9. Copy the temporary credentials when they are displayed. The password is shown in this dialog after creation.

Validation rules:

- Name, username, and email are required in the Admin Web Panel.
- Username must contain 3-64 letters, numbers, or underscores.
- Username must be unique, ignoring letter case.
- Email must be unique, ignoring letter case.
- A manually entered password must contain at least 8 characters.
- If no password is entered, the platform generates a temporary password.

Implementation note: the API can derive a missing username from the part of the email address before `@`, replacing unsupported characters with underscores. The Admin Web Panel normally does not use this fallback because its username field is required.

Users can authenticate with either their username or email address.

5.2 View Access

1. Open the user's actions menu.
2. Select **View Access**.

The dialog shows the username, email, role, status, assigned groups, and deployments inherited from those groups.

5.3 Edit User

1. Open the user's actions menu.
2. Select **Edit**.
3. Update the name, username, email, role, status, or group assignments.
4. Select **Save User**.

The same username and email uniqueness rules used during creation also apply to edits. Passwords are changed through **Reset Password**, not the edit form.

5.4 Disable User

1. Open the user's actions menu.
2. Select **Disable**.

3. Confirm the action.

Disabled users cannot authenticate. The current interface does not provide a separate re-enable action; an administrator can open **Edit**, change **Status** to **Active**, and save the user.

5.5 Reset User Password

1. Open the user's actions menu.
2. Select **Reset Password**.
3. Enter a new password of at least 8 characters.
4. Select **Reset Password**.
5. Copy the credentials shown after the reset.

5.6 Delete User

1. Open the user's actions menu.
2. Select **Delete**.
3. Confirm the permanent deletion.

Deleting a user cannot be undone from the Admin Web Panel. Prefer disabling an account when its history may still be needed.

6. Group and Deployment Access Management

Groups and their deployment grants are managed together in **Users & Permissions**. Users inherit access to all deployments selected for their groups.

6.1 Create Group

1. Go to **Users & Permissions**.
2. Select **New Group** or **Create Group**.
3. Enter a unique group name.
4. Expand **Members** and select any users to add.
5. Expand **Deployment Access** and select any deployments to grant.
6. Select **Save Group**.

Group names are required and unique.

The **Group Access** catalog is paginated at 4 groups per page. Member and deployment lists inside each group card are independently scrollable.

6.2 Manage Membership and Deployment Access

1. Find the group under **Group Access**.
2. Select **Manage**.
3. Add or remove users under **Members**.
4. Grant or revoke deployments under **Deployment Access**.
5. Select **Save Group**.

Saving applies the membership changes and then adds or removes deployment grants as needed. Granting an existing mapping returns a conflict at API level; revoking a mapping that does not exist returns not found. The checkbox-based interface normally avoids both conditions.

6.3 Delete Group

1. Find the group under **Group Access**.
2. Select **Manage**.
3. Select **Delete Group**.
4. Review and confirm the warning.

Deleting a group removes its user-membership rows and deployment-access grants. It does not delete the associated user accounts or deployments.

7. Deployment Management

7.1 Create Deployment

1. Go to **Deployments**.
2. Select **+ New Deployment**.
3. Enter a unique deployment name.
4. Optionally enter a logo URL and description.
5. Select **Create Deployment**.

Deployment names are required and unique, ignoring letter case.

7.2 View or Edit Deployment

1. Find the deployment in grid or list view.
2. Select **View** to review its metadata and versions, or select **Edit**.
3. Update the name, logo URL, or description.
4. Save the deployment.

The page supports search, status and channel filters, sorting, grid/list views, and pagination.

7.3 Archive, Restore, or Delete Deployment

Open the deployment's additional-actions menu and choose the required action:

- **Archive deployment** changes every non-deleted version in that deployment to **Archived**. A deployment without versions cannot be archived.
- **Restore draft** changes archived versions in that deployment back to **Draft**.
- **Delete deployment** asks for confirmation and removes the deployment and its associated version records from the platform database.

Package files are not documented as being removed by these actions. Use deployment-level lifecycle actions only when the whole package family should change state.

8. Version Management

8.1 Register Version

1. Go to **Versions**.
2. Select the target deployment.
3. Select **+ Register Version**.
4. Enter the version number.
5. Choose **Stable** or **Beta**.
6. Choose one package source:
    - **Package server folder**
    - **Register server archive**
    - **Upload local archive**

7. For **Package server folder**, prepare one deployment root containing the component folders and one root-level launch batch file, enter its absolute server path, enter the version number, and select **Inspect & prepare package**. For **Register server archive**, enter the existing ZIP/7z server path and select **Validate server archive**. For **Upload local archive**, select a prepared ZIP or 7z from the current computer, then select **Upload & validate archive**. The page displays transfer percentage, bytes, elapsed time, and estimated remaining time. The backend calculates SHA-256 while streaming the upload to disk, then checks its structure and launch script. Local archives transfer in resumable 64 MB chunks. If the network drops, the page retries from the last byte confirmed by the backend instead of restarting the complete file. After a browser refresh, select the same local file again to reconnect to its persisted upload session. Server-folder preparation and server-archive validation calculate the package checksum during this step.
8. Select the initial status: **Draft**, **Released**, or **Archived**.
9. Optionally enter a description.
10. Review the detected file, batch-script, size, and checksum metadata.
11. Select **Register version**.

The version number is required and must be unique within its deployment. Selecting **Cancel** discards the complete registration draft, including the package path, selected upload, prepared archive metadata, checksum, validation state, and description. Switching deployments also starts with a clean draft. Navigating to another admin page does not cancel preparation or registration. The current draft and phase are retained for the browser session and restored when returning to **Versions**. If registration finishes while another page is open, the registered version appears when the Versions list reloads. Closing or refreshing the browser pauses a local upload until the same file is selected again; server-side preparation jobs continue and are restored from the draft. After a server package is prepared or validated, registration reuses its calculated checksum instead of reading the complete archive a second time. The page enables registration only after preparation returns a generated archive path, file name, nonzero size, launch script, and checksum. The backend also rejects direct registration of an unprepared staging folder, preventing packaging from unexpectedly starting during **Register version**. Local archive registration is enabled only after upload and validation return the stored file identifier, nonzero size, launch script, and checksum. Register does not upload the file a second time.

8.2 Package Requirements

- Archives must be ZIP or 7z files.
- The package must contain a launch `.bat` file either at the archive root or inside its only top-level folder.
- A server path must exist and be inside the backend's configured package root.
- A server archive path must point to a file.
- A server staging-folder path must point to a non-empty directory containing a launch batch script.

- Select a deployment subfolder, not `PACKAGE_ROOT` itself; generated archives are written under `PACKAGE_ROOT/_generated`.
- ZIP archives are inspected directly.
- Reading or creating 7z archives requires `7z` or `7za` on the backend server.

For large Unreal deployments, prefer **Server staging folder** on the backend PC. When 7-Zip is available, the backend creates a generated 7z archive. Without 7-Zip, it creates a ZIP when the staging folder is within the built-in ZIP size limit.

Preparation always rebuilds the generated archive from the current staging folder and then calculates SHA-256. Duplicate preparation clicks or requests share one backend job. Large builds can still take several minutes depending on total bytes, file count, and disk speed. Remove runtime caches, logs, crash dumps, and temporary files before preparation. A failed job removes its partial temporary archive; select **Inspect & prepare package** again after correcting the reported error.

Preparation runs as a background backend job. The Version form reports:

- Current phase: scanning, archive creation, archive validation, or checksum
- Phase percentage when 7-Zip or checksum processing can measure it
- Elapsed time
- Estimated remaining time after measurable progress begins
- Processed and total bytes during checksum generation

The scanning phase shows an indeterminate progress bar because 7-Zip must first discover the package files before it can calculate a meaningful percentage. Admins may navigate to other portal pages and return to the Version page; the page reconnects to the same job. Starting the same deployment/version/path again returns the active job instead of launching another archive process.

8.3 Manage a Version

The version table provides these actions:

- Select the channel control to switch between **Stable** and **Beta**.
- Select the status control to choose **Draft**, **Released**, or **Archived**.
- Select **Release** to make a non-released version released.
- Select **Archive** to hide a non-archived version.
- Select **Restore draft** to return an archived version to draft.
- Select **View details** to review metadata and edit the description, channel, or status.

- Select **Delete** and confirm to remove the version record from active use.

Only **Released** versions are visible to authorized launcher users. Draft, archived, and deleted versions are hidden. Deleting a version does not delete its package file from the server.

9. Notifications, Logs, and Monitoring

9.1 Notifications

Notifications are created for active managed administrators when:

- A deployment is created, archived, restored, or deleted
- A version is registered, updated, released, archived, restored, or deleted
- A launcher user requests a download
- The launcher submits an error report

Use the notification bell for quick triage. Its menu supports **Mark all read**, deleting individual notifications, and **Clear all**. Clear all requires confirmation and permanently removes every notification belonging to the signed-in administrator.

From the full Notifications page, filter **All**, **Unread**, or **Read** notifications, mark individual or all notifications as read, and delete resolved notifications individually.

9.2 Download Logs

Go to **Download Logs** to:

- View download time, user, username, deployment, version, channel, and IP address
- Filter the list by deployment
- Export the current deployment scope as a CSV file

9.3 Launcher Reports

Go to **Launcher Reports** to review failures submitted by signed-in launcher users. Reports can be filtered by deployment and by:

- **Download / Install**
- **Deployment Launch**
- **Launcher Update**

Select **View** to inspect the report time, user, launcher version, machine, operating system, deployment/version context, and diagnostic details.

10. Settings and Maintenance Mode

The **Settings** page contains four tabs:

- **General**: view the Admin Web Panel version and edit the product name and support email.
- **Server**: view API/download URLs and run **Test Connection** to check the database URL, token secrets, package root, upload/download storage, backend port, and 7-Zip readiness.
- **Security**: view the signed-in username and role, or sign out. The displayed **Change Password** action is not implemented yet; reset an administrator's password from **Users & Permissions** instead.
- **Maintenance**: enable maintenance mode, set its message, open Download Logs for export, reset settings to their defaults, and save maintenance settings.

Maintenance mode blocks non-admin download-manager listing, session, and file streaming operations. Administrators remain able to use the system. Enter a maintenance message before saving if launcher users need a specific explanation.

11. Admin Best Practices

- Use group-based access rather than user-by-user exception mappings.
- Keep deployment names and version numbers consistent.
- Keep new versions in **Draft** until their package and access have been tested.
- Archive obsolete versions instead of deleting records when history matters.
- Validate a release with a real active launcher test account after granting group access.
- Review notifications, download logs, and launcher reports after releases.
- Copy generated or reset credentials before closing the credentials dialog.

12. Common Admin Issues

12.1 User cannot sign in

Check:

- The username or email and password are correct
- The account status is **Active**
- A temporary rate-limit cooldown is not in effect

- Maintenance mode is not blocking the user's launcher operation after login

12.2 User cannot see deployment

Check:

- The user is active
- The user belongs to at least one group
- At least one of those groups has access to the deployment
- The target version is **Released**

12.3 Version registration or validation fails

Check:

- The source path exists on the backend server
- The source is within the configured package root
- The selected source type matches a file, folder, or upload
- The archive is ZIP or 7z
- The package contains a launch `.bat` at its root or inside its only top-level folder
- 7-Zip is available when reading or creating a 7z package
- The version number is not already registered for that deployment

12.4 Notifications are empty

Check:

- A notification-producing event has occurred
- The signed-in account is an active managed user with the **Admin** role
- The backend database is reachable

12.5 Server readiness shows warnings or offline

Open **Settings > Server**, select **Test Connection**, and review each check. Correct required database or package-root failures before publishing packages. Treat a missing 7-Zip check as a warning unless the workflow requires 7z inspection or large staging-folder packaging.

Code and Documentation Audit (2026-07-27)

Scope

This audit compared the active project documentation with:

- Backend routes, controllers, services, repositories, Prisma schema, and environment-variable usage
- Frontend routes, API client, admin pages, and authentication guard
- Launcher API client, download/install flow, settings, prerequisite checks, self-update, error reporting, tests, and installer build script
- Nginx configuration and package-delivery path rules
- Package manifests and example environment files

Uploaded Markdown files under `backend/storage/downloads` and the original HTML project brief are package/source artifacts, not maintained runtime documentation, and were not rewritten.

Documentation Updated

- Root, backend, frontend, infrastructure, and admin/operations documentation now uses current UI labels, routes, commands, package-source behavior, and environment variables.
- Nginx documentation now distinguishes `PACKAGE_ROOT` validation from `DOWNLOAD_ROOT` acceleration and recommends a shared private directory.
- Architecture diagrams now show token validation in the backend before direct streaming or `X-Accel-Redirect`.
- Requirements, design, and task files now state whether they are acceptance criteria, target design, or planning history rather than current runtime verification.
- Example database credentials were replaced with placeholders.
- User-management documentation now matches the six-user page size, shared user/email/group search, and four-group Group Access pagination.
- Group-management documentation now covers confirmed deletion and accurately states that membership/access mappings cascade while users and deployments remain.
- Notification documentation now distinguishes dropdown **Clear all** from individual deletion on the full Notifications page.

- Launcher documentation now reflects the deployment-first catalog, side-by- side version actions, and branded maintenance-mode dialog.

Executable Verification

Run from the repository root unless noted:

```
node --test backend\test\authMiddleware.test.js backend\test\downloadManagerService.test.js
dotnet test launcher\Launcher.Tests\Launcher.Tests.csproj -p:Configuration=Debug
```

Run from `backend`:

```
npx.cmd prisma validate
```

Run from `frontend`:

```
npm.cmd run build
```

Results:

- Backend tests: 17 passed
- Launcher tests: 14 passed
- Prisma schema validation: passed
- Frontend production build: passed

Authorization Follow-up

The authorization gap found during this audit was fixed on 2026-07-27:

- `requireAdmin` middleware now protects all `/api/users` user/group routes.
- The frontend portal guard now requires an Admin role claim in the current, unexpired JWT and clears non-admin sessions.
- Middleware tests confirm Admin access and HTTP 403 rejection for non-admin or missing-role requests.

Deployment/version writes, settings, and admin reporting retain their existing handler-level Admin checks. A future cleanup can centralize those checks on route middleware as well.

Remaining Code Gaps

Test and CI coverage

- The backend has no `npm test` script even though Node test files exist.
- Full user, group, deployment, and route-level authorization flows still need API integration tests beyond the middleware unit coverage.
- Installer upgrade preservation and full hosted Nginx delivery still need environment-level smoke tests.

Documentation Boundaries

- `design.md` remains an explicitly labeled original target design. Its `/api/admin/*` and proposed launcher endpoint tables are intentionally historical and are not the runtime API reference.
- `requirements.md` is acceptance criteria rather than implementation proof. Product behavior changed during this review: user pages now contain 6 rows, groups can be deleted with relational cleanup, and the launcher catalog is grouped by deployment rather than separated into channel sections. Those criteria were updated to match the approved behavior.
- Uploaded Markdown under `backend/storage/downloads` remains package content and is not maintained as project documentation.

Full Requirements Audit (2026-07-23)

This is a broad implementation audit across frontend, backend, and launcher. It is based on code inspection and targeted validation commands, not exhaustive end-to-end test execution for every acceptance criterion.

Validation Signals Used

- Launcher build passed: dotnet build launcher/Launcher.csproj -p:Configuration=Debug
- Backend download-manager test suite passed: node --test backend/test/downloadManagerService.test.js
- Frontend/admin and backend routes/services reviewed for user, deployment, access, logging, and launcher flows.

Requirement Status Summary

Requirement	Status	Notes
1. Admin Authentication	Partial	Login, JWT, and rate limiting are implemented. Full verification of exact 24h expiry and all UI field constraints needs dedicated API/UI tests.
2. User Management	Partial	CRUD, disable/reset flows are present. Full validation of all username/password rule edge cases and pagination contract requires focused tests.
3. Deployment Management	Partial	Deployment create/rename/list behavior appears implemented. Conflict and full admin UI behavior should be verified with integration tests.
4. Version Management	Partial	Upload/server path/staging folder flows and metadata are present. Full grouped UI behavior and all failure message paths need E2E confirmation.
5. Group-Based Access Control	Partial	Group/user/deployment access plumbing exists. Timing guarantee (within 5 seconds) needs measured integration verification.
6. Launcher Authentication	Implemented (high confidence)	Credential-store persistence, expired-token handling, sign-out clear, disabled account messaging, and 429 retry-after behavior are implemented.
7. Launcher Discovery	Implemented (high confidence)	Auth-gated item retrieval, released-only visibility behavior, 5-minute polling, and manual refresh are implemented.
8. Package Download	Implemented (high confidence)	Session token flow, resumable range downloads, pause/resume/cancel, progress telemetry, disk checks, and token refresh behavior are implemented.
9. Integrity and Extraction	Implemented (high confidence)	SHA-256 verification, retry-on-mismatch, extraction, and missing-checksum block are implemented.
10. Side-by-Side Installation	Implemented (high confidence)	Per-version install folders, installed-state checks, duplicate install guard, and per-version uninstall behavior are implemented.
11. Install Location Configuration	Implemented (high confidence)	Path validation, create-if-missing prompt, writability checks, and persistence are implemented.
12. Installed Version Access	Implemented (high confidence)	Open folder behavior and missing-folder error handling are implemented.
13. Error Handling and Feedback	Partial	Friendly error mapping and retry/pause handling are implemented. Some exact copy and all edge-case dialogs should be confirmed via UX test pass.
14. Launcher Branding	Implemented (high confidence)	Config-driven logo, format/size checks, fallback behavior, and packaging workflow are implemented.
15. Download Token Security	Implemented (aligned)	Token validation/scoping and refresh behavior are implemented and aligned to the current one-hour policy in requirements.md.
16. Installer and Updates	Partial	Inno Setup packaging, shortcuts, self-contained publish, and update endpoint flow exist. Upgrade-preservation behavior should be validated with install/upgrade smoke tests.

Newly Hardened Download Reliability (This Update)

- Adaptive stream selection now tunes active stream count while staying inside the 4-16 range.
- Retry timing now uses jittered exponential backoff to reduce reconnect thrashing on unstable links.
- Existing resume logic, chunk repair pass, and token refresh behavior remain in place.

Remaining Gaps Before Final Sign-Off

1. Execute full end-to-end acceptance tests for Requirements 1-5 and 16 with scripted evidence.
2. Add automated regression tests for launcher network failure scenarios (packet loss, reconnect churn, long-haul downloads).
3. Add backend/frontend CI checks to enforce auth/access/validation contracts on every change.

Implementation Verification (2026-07-24)

This document records the current launcher download-manager implementation status versus requirements, based on code review plus executable checks.

Revalidated on 2026-07-27. The current backend suites contain 17 passing tests, including Admin authorization middleware coverage; the launcher suite contains 14 passing tests.

Verification Run

- Launcher build: `dotnet build .\launcher\Launcher.csproj -p:Configuration=Debug` (pass)
- Backend authorization and package/download-manager tests: `node --test backend\test\authMiddleware.test.js backend\test\downloadManagerService.test.js` (17/17 pass)
- Launcher policy tests: `dotnet test launcher\Launcher.Tests\Launcher.Tests.csproj -p:Configuration=Debug` (14/14 pass)
- Backend Prisma schema validation from `backend`: `npx.cmd prisma validate` (pass)
- Frontend admin panel build from `frontend`: `npm.cmd run build` (pass)

Requirement Coverage Snapshot

Confirmed Implemented (launcher + download-manager scope)

- Requirement 6.3: launcher blocks entry when credential-store save fails.
- Requirement 6.6: expired stored JWT is cleared on startup.
- Requirement 6.7: invalid token without `exp` is not pre-cleared; token is attempted and handled on API call.
- Requirement 6.8: sign-out clears token from Windows Credential Manager.
- Requirement 6.10: HTTP 429 login response disables sign-in flow for `Retry-After` duration (default 60s).
- Requirement 7.5: launcher polling timer runs every 5 minutes.
- Requirement 7.6: manual refresh is available.
- Requirement 8.1: launcher creates a download session/token before transfer.
- Requirement 8.4 and 8.5: download stream count is clamped between 4 and 16.

- Requirement 8.6 and 8.7: resume uses persisted `.part` chunk files and saved state.
- Requirement 8.8 and 8.9: Pause/Resume/Cancel controls exist; cancel deletes partial artifacts.
- Requirement 8.10: after bytes reach 100%, the launcher enters an explicit verifying/extracting phase instead of continuing to show the stale download state.
- Requirement 8.11 and 8.12: free-space check runs before transfer and reports required vs available space.
- Requirement 8.14: low free space mid-download pauses transfer and reports shortfall.
- Requirement 9.2 and 9.3: SHA-256 verification is enforced after download; failed checksum retries up to 3 times.
- Requirement 9.4: ZIP extraction uses built-in .NET extraction; 7z extraction uses `7z.exe` or `7za.exe` beside the launcher or on `PATH`.
- Requirement 9.7: launcher blocks installation when checksum metadata is missing.

Implemented And Aligned Requirements

- Requirement 15.3: current implementation uses 1-hour download-manager tokens and refreshes at 55 minutes or when token expiry is within 60 seconds.

Notes

- This verification covers launcher authentication/download-manager behavior and its paired backend token/session logic.
- Version records use `draft`, `released`, `archived`, and `deleted` states. Download sessions separately use runtime states such as `paused`, `canceled`, `failed`, and `completed`.
- It is not a full-system audit of every requirement in `requirements.md`.

Building the VIZZIO Launcher Installer

Prerequisites

Install the following on the build computer:

- .NET 8 SDK
- Inno Setup 6
- 7-Zip

Inno Setup should be installed at:

```
C:\Program Files (x86)\Inno Setup 6\ISCC.exe
```

Backend Configuration

Before building, confirm the launcher's default API endpoint in:

```
launcher\DownloadManagerApiClient.cs
```

The production endpoint is:

```
https://vzdeployment.hardyhutajaya.com/api
```

Do not use `localhost` when distributing the launcher to other computers.

Build Command

Open PowerShell in the project root:

```
cd "C:\Users\User\Desktop\VIZZIO_Deployment_Platform"
```

Run:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File ".\scripts\build_launcher_installer.ps1" `
  -Version "0.1.0" `
  -InnoCompiler "C:\Program Files (x86)\Inno Setup 6\ISCC.exe"
```

The execution-policy bypass applies only to this PowerShell process and does not permanently change the system policy.

Installer Output

After a successful build, the installer is generated at:

```
installer\artifacts\VIZZIO-Launcher-Setup-0.1.0.exe
```

Distribute this `.exe` to users. The installer includes the .NET runtime and 7-Zip extraction support, so users do not need to install those dependencies separately.

Inno Setup is required only on the computer used to build the installer.

Launcher self-update

The launcher checks the backend after a user signs in or a saved session is restored:

```
GET /api/launcher/update?currentVersion=0.1.0
```

The backend response is controlled with environment variables:

```
LAUNCHER_LATEST_VERSION=0.2.0
LAUNCHER_DOWNLOAD_URL=https://example.com/VIZZIOLauncherSetup-0.2.0.exe
LAUNCHER_RELEASE_NOTES=Bug fixes and launcher improvements.
LAUNCHER_UPDATE_REQUIRED=false
```

If `LAUNCHER_LATEST_VERSION` is newer than the launcher's assembly version, the launcher prompts the user to install the update. If the user accepts, it downloads the installer to a temporary folder, starts it, and closes the current launcher so the installer can replace the app files.

If `LAUNCHER_UPDATE_REQUIRED=true`, declining the update closes the launcher.

Launcher error reporting

The Windows launcher uploads diagnostic reports to the backend when a signed-in user hits a launcher-side failure.

Current report sources:

- Download failures, including insufficient disk space.
- Launch failures, including missing `Launch.bat`, missing package content, missing prerequisites, and declared port conflicts.
- Launcher self-update check failures after sign-in.

Reports are written as JSON files by the backend. The default folder is:

```
backend/storage/launcher-error-reports
```

Set this environment variable in production to keep reports on persistent storage:

```
LAUNCHER_ERROR_REPORT_ROOT=C:\VIZZIO\launcher-error-reports
```

Each report includes the signed-in user identity, launcher version, machine and OS details, the deployment/version when available, the user-facing error message, request metadata, and the tail of the local launcher log.

Admins can review reports from **Launcher Reports** in the sidebar.

Endpoint:

```
POST /api/download-manager/error-reports
```

The endpoint requires the normal launcher bearer token. If the user is not signed in, the launcher keeps the error local and skips upload.

Operations Guide: Publishing a New Version

1. Purpose

This guide defines the standard operating procedure for publishing a new deployment version from package intake through launcher visibility verification.

2. Preconditions

- Admin account with deployment/version permissions
- Backend API running and database healthy
- Package source prepared:
  - archive file, or
  - server staging folder
- Expected launch batch script is included at the archive root or inside the only top-level folder
- Target deployment exists (or will be created in this process)

3. Release Checklist

1. Confirm package identity and version number.
2. Confirm deployment target name.
3. Confirm package source path or file.
4. Confirm channel assignment (Stable or Beta).
5. Confirm status assignment (Draft, Released, or Archived).
6. Confirm target group access plan.
7. Confirm rollback option (previous released version still available).
8. Confirm notification and monitoring plan.

4. Publish Workflow

Step 1: Log in to Admin Panel

Authenticate as an administrator and open **Deployments**.

Step 2: Select or Create Deployment

- Use existing deployment, or
- Create a new deployment with a unique name.

Step 3: Register Version

Open **Versions**, choose the deployment, and select **+ Register Version**. Enter the version number before preparing a server folder because it is used for the generated archive name.

When adding a version, choose one package source:

- Upload and validate an archive from the local machine
- Register existing server archive path
- Register server staging folder path (system archives folder into package)

ZIP and 7z archives must contain a launch `.bat` file at the archive root or inside one wrapper folder. Both of these are accepted:

```
SICC-v2.zip
  Launch.bat
  Windows/
```

```
SICC-v2.zip
  SICC/
    Launch.bat
    Windows/
```

Deeply nested or ambiguous layouts, such as `Builds/SICC/Launch.bat`, are rejected because the launcher expects the script at the installed package root after extraction. 7z validation requires `7z` or `7za` on the backend server.

For 50-60 GiB Unreal deployments, prefer the server staging-folder flow. With 7-Zip installed on the backend PC, staging folders are converted into generated `.7z` packages using store mode instead of relying on small built-in ZIP packaging. Remove runtime caches, logs, crash dumps, and temporary files from the staging folder before preparation; every included file must be scanned and written to the archive. Select the

individual deployment folder, never `PACKAGE_ROOT` itself, because generated packages are written below `PACKAGE_ROOT/_generated`.

Set:

- Version number
- Channel: Stable or Beta
- Initial status: Draft, Released, or Archived

Deployment-level Archive/Restore/Delete actions are available on the Deployments page and apply to every version in that deployment. Version-level Archive/Restore/Delete actions are available on the Versions page and apply only to the selected version.

Use Draft while reviewing package metadata. Use Released only when the package should appear in the launcher for authorized users. Use Archived to keep the record hidden from launcher users.

Step 4: Validate Version Metadata

Verify the newly added version includes:

- Expected package artifact path/name
- Size metadata
- Checksum calculated during preparation or validation
- Correct channel and status

Validation checks package shape and launch-script presence. Server-folder preparation creates a fresh archive and calculates SHA-256. Existing server-archive validation also calculates SHA-256. Registration reuses that checksum and does not read the entire archive again. Registration rejects raw staging-folder paths and server archives that have not returned validation metadata. A successful preparation must display the generated file name, nonzero size, detected launch script, and SHA-256 checksum before **Register version** is available.

For a local archive, select the file and run **Upload & validate archive** before registration. Upload progress is measured in the browser. The backend streams the file to disk and calculates SHA-256 in the same pass, then validates the archive layout and launch script. Registration reuses that stored package record and does not transfer the archive again.

Only one preparation job runs for a deployment/version output at a time within the backend process. Duplicate requests wait for the same job. Packaging and checksum time still scale with package size, file count, and storage speed; this is expected for large builds and is not an instant metadata operation.

Preparation is exposed as a background job. The admin page polls its authenticated status endpoint and displays scanning, archive creation, checksum, elapsed time, and ETA when a percentage is available.

Navigating within the portal does not stop the job. Parallel archive creation is intentionally not used because multiple writers compete for the same source and destination disk and were observed to reduce throughput.

If preparation fails or is interrupted, retry **Inspect & prepare package**. Failed jobs remove their temporary archive file, while an older completed archive remains available until a fresh replacement has completed.

Step 5: Grant Access

Open **Users & Permissions**, find or create the intended group, select **Manage**, choose the deployment under **Deployment Access**, and save the group.

Step 6: Functional Verification

Use a real launcher test user account in target group and validate:

- Deployment appears in library
- Target version appears in correct channel
- Archived versions are hidden
- Download session can start
- Download logs register activity
- Admin notification appears for the version/deployment change or download request
- Launcher Reports stays clear of new launch, prerequisite, install, or download failures

Step 7: Release Communication

Record and share:

- Deployment name
- Version number
- Channel and status
- Accessed groups
- Release time
- Known caveats

5. Rollback Procedure

If release is bad:

1. Set bad version status to Archived.
2. Ensure prior known-good version is Released.
3. Notify stakeholders.
4. Document root cause and corrective action.

6. Naming and Versioning Guidelines

- Keep deployment names stable and descriptive.
- Use consistent semantic-style version numbering where possible.
- Avoid reusing version numbers for different package contents.

7. Operational Safety Rules

- Do not publish directly to broad groups without staged validation.
- Validate access changes after group updates.
- Avoid deleting records needed for traceability; prefer archive/disable states.
- Run smoke checks immediately after each release.

8. Troubleshooting During Publish

8.1 Path not found or invalid source

- Verify source exists on server.
- Verify source is under configured package root.
- Verify staging folder contains expected launch script.

8.2 Users cannot see released version

- Confirm version status is Released.
- Confirm deployment access grants exist for user groups.
- Confirm user group membership.
- Trigger launcher manual refresh and retest.

8.3 Download starts but no logs appear

- Confirm active backend instance and port.
- Confirm download session creation path succeeded.
- Confirm download log insert path and database connectivity.

8.4 Notifications do not appear

- Confirm the admin account is active and has Admin role in the managed users table.
- Confirm the event happened after notification support was enabled.
- Confirm backend notification writes are not failing in the backend terminal logs.
- Download-request notifications are created when the launcher creates a managed download session.

9. Post-Release Audit

Capture these fields in release notes:

- Deployment
- Version
- Channel
- Status
- Source type
- Groups granted
- Verification account used
- Verification timestamp
- Result
- Notification/log/report review result

Configuration and Production Handover

1. Purpose

This is the authoritative configuration checklist for local acceptance testing, supervisor review, and production deployment of the VIZZIO Deployment Platform. Do not promote a build until every item in the relevant checklist passes.

2. Components and URL Ownership

Component	Local testing	Production
Backend API	`http://localhost:4000/api`	`https://<public-host>/api`
Backend health	`http://localhost:4000/api/health`	`https://<public-host>/api/health`
Admin frontend	Vite development URL	Public frontend hostname
Frontend API/download paths	Same-origin `/api`, `/downloads`	Same-origin `/api`, `/downloads`
Launcher API	Built as `http://localhost:4000/api`	Build with `-ApiBaseUrl "https://<public-host>/api"`
PostgreSQL	`localhost:5432`	Private database address

The launcher first uses the `VIZZIO_API_BASE` environment variable when it is present. Otherwise it uses the URL stamped into the launcher assembly at build time. Local builds default to `http://localhost:4000/api`.

3. Local Acceptance-Test Configuration

3.1 Backend

Create `backend/.env` from `backend/.env.example` and use:

```
DATABASE_URL=postgresql://postgres:<password>@localhost:5432/vizzio_deployment
NODE_ENV=development
JWT_SECRET=<local-secret>
DOWNLOAD_SECRET=<different-local-secret>
DOWNLOAD_MANAGER_SECRET=<third-local-secret>
PORT=4000

PACKAGE_ROOT=C:\VIZZIO\packages
PACKAGE_UPLOAD_ROOT=C:\VIZZIO\uploads
PACKAGE_UPLOAD_MAX_BYTES=85899345920
UPLOAD_SESSION_RETENTION_MS=604800000

DOWNLOAD_DELIVERY_MODE=node
DOWNLOAD_ROOT=C:\VIZZIO\packages
DOWNLOAD_ACCEL_PREFIX=/_vizzio_downloads

HTTP_REQUEST_TIMEOUT_MS=0
HTTP_HEADERS_TIMEOUT_MS=60000
HTTP_KEEP_ALIVE_TIMEOUT_MS=5000

LAUNCHER_ERROR_REPORT_ROOT=C:\VIZZIO\launcher-error-reports
LAUNCHER_LATEST_VERSION=0.1.0
LAUNCHER_DOWNLOAD_URL=
LAUNCHER_RELEASE_NOTES=Local acceptance build
LAUNCHER_UPDATE_REQUIRED=false
ENABLE_DEMO_USERS=false
```

Create these directories and grant the backend service account read/write permission:

```
C:\VIZZIO\packages
C:\VIZZIO\uploads
C:\VIZZIO\uploads\.sessions
C:\VIZZIO\launcher-error-reports
```

Install 7-Zip and verify `C:\Program Files\7-Zip\7z.exe` exists.

3.2 Database and backend startup

```
cd backend
npm install
npx prisma generate
npx prisma migrate deploy
node src/index.js
```

Verify:

```
Invoke-RestMethod http://localhost:4000/api/health
node --test test/*.test.js
```

Expected health result:

```
{"status":"ok"}
```

3.3 Admin frontend

No frontend `.env` is required when frontend and backend share a hostname. During `npm run dev`, Vite proxies `/api` and `/downloads` to port 4000.

```
cd frontend
npm install
npm run dev
```

For an explicit local override:

```
VITE_API_BASE=http://localhost:4000/api
VITE_DOWNLOAD_BASE=http://localhost:4000/downloads
```

3.4 Local launcher

Build and run:

```
dotnet run --project launcher\Launcher.csproj
```

The default local API is `http://localhost:4000/api`. An override for one terminal session is:

```
$env:VIZZIO_API_BASE = "http://localhost:4000/api"
dotnet run --project launcher\Launcher.csproj
```

Clear an old override before testing the build-stamped endpoint:

```
Remove-Item Env:VIZZIO_API_BASE -ErrorAction SilentlyContinue
```

4. Local End-to-End Acceptance Checklist

1. Backend readiness reports database, secrets, storage roots, and 7-Zip ready.
2. Admin can log in and create a deployment.
3. Local archive upload can pause/interruption-retry and resume from its confirmed 64 MB chunk offset.
4. Existing server ZIP/7z validation returns file, launch script, size, and SHA-256.
5. Server staging-folder preparation creates a generated archive and SHA-256.
6. Final registration does not upload or checksum an unchanged package again.
7. Released versions appear only for launcher users with group access.
8. Launcher creates a download session and performs parallel range downloads.
9. Pausing/restarting the launcher resumes `.part` files.
10. SHA-256 verification and extraction succeed.
11. The detected root-level launch batch script starts successfully.
12. Download logs, notifications, and launcher error reporting are visible in the admin panel.

5. Production Backend Configuration

Use three independently generated cryptographic secrets. Never copy sample or local secrets into production. Rotating them invalidates active sessions and download tokens.

```
DATABASE_URL=postgresql://<user>:<password>@<private-db-host>:5432/<database>
NODE_ENV=production
JWT_SECRET=<random-production-secret-1>
DOWNLOAD_SECRET=<random-production-secret-2>
DOWNLOAD_MANAGER_SECRET=<random-production-secret-3>
PORT=4000

PACKAGE_ROOT=<durable-package-directory>
PACKAGE_UPLOAD_ROOT=<durable-upload-directory>
PACKAGE_UPLOAD_MAX_BYTES=85899345920
UPLOAD_SESSION_RETENTION_MS=604800000

HTTP_REQUEST_TIMEOUT_MS=0
HTTP_HEADERS_TIMEOUT_MS=60000
HTTP_KEEP_ALIVE_TIMEOUT_MS=5000

LAUNCHER_ERROR_REPORT_ROOT=<durable-report-directory>
LAUNCHER_LATEST_VERSION=<released-launcher-version>
LAUNCHER_DOWNLOAD_URL=https://<public-host>/<installer-file>
LAUNCHER_RELEASE_NOTES=<release-summary>
LAUNCHER_UPDATE_REQUIRED=false
ENABLE_DEMO_USERS=false
```

5.1 Windows/Cloudflare with Node file delivery

```
DOWNLOAD_DELIVERY_MODE=node
DOWNLOAD_ROOT=C:\VIZZIO\packages
DOWNLOAD_ACCEL_PREFIX=/_vizzio_downloads
```

Cloudflare must forward the public hostname to the backend/frontend reverse proxy. Confirm its upload-size and request-duration limits are compatible with the intended package size. Resumable 64 MB upload chunks avoid one multi-hour request, but the proxy must permit each chunk.

5.2 Linux/Nginx accelerated delivery

```
PACKAGE_ROOT=/var/www/vizzio/builds
PACKAGE_UPLOAD_ROOT=/var/www/vizzio/uploads
DOWNLOAD_ROOT=/var/www/vizzio/builds
DOWNLOAD_DELIVERY_MODE=nginx
DOWNLOAD_ACCEL_PREFIX=/_vizzio_downloads
```

Deploy `infra/nginx.conf` after replacing `server_name` and confirming its filesystem alias. The supplied configuration includes:

- `client_max_body_size 80g`
- `proxy_request_buffering off`
- 24-hour proxy send/read timeouts
- internal `X-Accel-Redirect` delivery
- range-friendly file serving

Never select Nginx mode on a Windows host that is not actually behind the configured Nginx instance.

6. Production Frontend

Preferred topology: serve frontend and API through one public hostname. Leave `VITE_API_BASE` and `VITE_DOWNLOAD_BASE` unset; the compiled frontend uses same-origin `/api` and `/downloads`.

If separate hostnames are required, set both variables before building:

```
VITE_API_BASE=https://api.example.com/api
VITE_DOWNLOAD_BASE=https://api.example.com/downloads
```

Build:

```
cd frontend
npm ci
npm run build
```

Never ship a production bundle containing `http://localhost:4000`.

7. Production Launcher and Installer

Local installer:

```
.\scripts\build_launcher_installer.ps1 `
  -Version "0.1.0" `
  -ApiBaseUrl "http://localhost:4000/api" `
  -InnoCompiler "C:\Program Files (x86)\Inno Setup 6\ISCC.exe"
```

Production installer:

```
.\scripts\build_launcher_installer.ps1 `
  -Version "0.1.0" `
  -ApiBaseUrl "https://<public-host>/api" `
  -InnoCompiler "C:\Program Files (x86)\Inno Setup 6\ISCC.exe"
```

The script bundles the self-contained .NET runtime, branding files, and 7-Zip. The output is `installer/artifacts/VIZZIO-Launcher-Setup-<version>.exe`.

Before distribution:

1. Install on a clean Windows test account.
2. Confirm the launcher Settings page displays the intended API endpoint.
3. Confirm the binary does not contain the wrong local/production endpoint.
4. Log in with a non-admin acceptance account.
5. Download, pause, resume, verify, extract, launch, and uninstall one package.
6. Confirm launcher update metadata and installer URL are reachable.

`VIZZIO_API_BASE` can override an installed launcher for diagnostics, but a normal production deployment should rely on the build-stamped URL.

8. Production Security and Operations

- Store `.env` and database credentials outside source control.
- Give package/upload directories only the permissions required by the backend service account.
- Terminate public traffic with valid HTTPS.
- Restrict PostgreSQL to private hosts.
- Back up PostgreSQL, package artifacts, upload manifests, and configuration.
- Monitor disk capacity for both archives and extracted-size estimates.
- Run one backend instance unless upload-session and preparation-job coordination is moved to shared storage.
- Keep `ENABLE_DEMO_USERS=false`.
- Review launcher reports, failed downloads, and authentication failures.

- Rotate secrets through an approved maintenance window.

9. Production Promotion Checklist

```
cd backend
npx prisma validate
npx prisma migrate deploy
node --test test/*.test.js

cd ..\frontend
npm run build

cd ..\launcher
dotnet test Launcher.Tests\Launcher.Tests.csproj --configuration Release
dotnet build Launcher.csproj --configuration Release
```

Then verify:

- Public `/api/health` returns HTTP 200.
- Unauthenticated `/api/download-manager/items` returns HTTP 401.
- Admin Settings readiness has no required errors.
- Frontend bundle contains no localhost production endpoint.
- Launcher assembly contains the intended production endpoint.
- One complete real-user download and launch passes.

10. Supervisor Sign-Off Record

Record:

Field	Value
Environment	Local acceptance / Staging / Production
Backend commit	
Frontend build timestamp	
Launcher version	
Public hostname	
Delivery mode	Node / Nginx
Package root	
Upload root	
Database migration	Pass / Fail
Backend tests	Pass / Fail
Frontend build	Pass / Fail
Launcher tests/build	Pass / Fail
End-to-end package	
Download/resume/checksum/extract/launch	Pass / Fail
Reviewer	
Date	
Notes	

Handover Document

1. System Summary

VIZZIO Deployment Platform is composed of:

- Backend API: Node.js + Express + Prisma + PostgreSQL
- Admin Portal: React + Vite
- Windows Launcher: .NET 8 WPF
- Delivery Layer: Node streaming and optional Nginx accelerated delivery

The platform enables controlled software release and distribution with group-based access, resumable downloads, and integrity checks.

2. Repository Ownership Areas

- backend: APIs, business logic, data model, download sessions, security
- frontend: admin UX, auth guard behavior, management pages
- launcher: end-user discovery/download/install experience
- infra: nginx config for reverse proxy and file delivery
- scripts and installer: packaging and distribution tooling

3. Runtime Topology

- PostgreSQL database for persistent state
- Backend API serving admin and launcher endpoints
- Optional Nginx path for large-file range delivery
- Launcher clients authenticating and downloading package artifacts

4. Critical Business Flows

- Admin auth and session lifecycle

- User/group/access management
- Deployment/version creation and release state transitions
- Launcher authentication and deployment discovery
- Download session creation, tokenized range fetches, checksum verification, extraction

5. Security Controls

- bcrypt hashing for user/admin passwords
- JWT auth for admin and launcher users
- Short-lived signed download tokens
- Login rate limiting to reduce brute-force attempts
- Package root and path safety validation
- Maintenance mode controls
- Admin-role middleware on all user/group management routes
- Admin-role validation in the frontend portal guard

6. Key Operational Procedures

- Publishing new versions: see docs/operations-publishing-guide.md
- Admin operations: see docs/admin-user-guide.md
- Architecture reference: see docs/architecture.md and README diagrams
- Complete local/production configuration and supervisor sign-off: `docs/configuration-and-production-handover.md`

7. Build and Validation Commands

Frontend

```
cd frontend
npm run build
```

Launcher

```
dotnet build launcher\Launcher.csproj -p:Configuration=Debug
```

Backend

```
cd backend
npm install
npm run prisma:generate
npm run prisma:migrate
npm run dev
```

8. Environment and Configuration

- backend/.env for API, DB, token, and package delivery settings
- frontend/.env for API and download base URLs
- launcher branding and runtime settings for client presentation

Use `backend/.env.example` and `frontend/.env.example` as the variable inventories. Replace every sample secret and set `ENABLE_DEMO_USERS=false` for hosted environments.

9. Known Risks and Guardrails

- Port conflicts in local development causing false negatives
- Large transfer reliability under unstable networks
- Token expiry during prolonged download
- Inconsistent test coverage across services

Guardrails:

- Single active backend instance in local validation
- Release smoke checks with real launcher account
- Post-release download log verification

10. Recommended Next Engineering Steps

- Add unified CI checks for backend, frontend, and launcher builds
- Expand backend automated tests beyond current service scope
- Add scripted health checks for local and deployment environments
- Formalize release note template and runbook enforcement

11. Support Handoff Notes

When transferring ownership:

1. Share environment secrets via approved secure channel only.
2. Walk through publish and rollback flow live.
3. Validate one end-to-end release in recipient environment.
4. Confirm access to database backups and server package storage.
5. Confirm installer build prerequisites on maintainer machine.